

Definición: Explica la diferencia entre un objeto y una clase usando una analogía diferente a la de "molde de galletas" o "lámpara".

Clase: es el diseño o la descripción general de algo. Define qué atributos y comportamientos tendrán sus instancias.

Objeto: es una instancia concreta creada a partir de una clase, con valores específicos en sus atributos.

Imagina un menú de restaurante y un platillo servido.

El menú describe los platillos disponibles, sus ingredientes y preparación → eso sería la clase.

El platillo servido es una comida real preparada para un cliente, con características concretas como tamaño, ingredientes extra o nivel de picante → eso sería el objeto.

Por ejemplo, la clase Pizza define atributos como tamaño y ingredientes, mientras que un objeto puede ser “Pizza grande de pepperoni con extra queso”.

Encapsulamiento: Si tienes un objeto CuentaBancaria, ¿por qué el atributo saldo no debería ser público?

El atributo saldo de una CuentaBancaria no debería ser público porque el encapsulamiento protege la integridad de los datos.

Si fuera público, cualquier parte del programa podría modificarlo directamente:

```
cuenta.saldo = -100000;
```

Eso podría generar estados inválidos o inseguros. En cambio, el saldo debe ser `private` y modificarse solo mediante métodos controlados, como `depositar()` y `retirar()`, que validen reglas de negocio:

- No permitir saldos negativos.
- Registrar movimientos.
- Verificar límites o permisos.

El encapsulamiento mejora la seguridad, la consistencia y el mantenimiento del sistema.

UML (Teoría): ¿Qué representan las flechas en un diagrama de clases? Explica la diferencia entre una línea continua y una línea con un diamante al final (composición).

En UML, las líneas y flechas indican relaciones entre clases.

Línea continua (asociación)

Representa que una clase conoce o usa a otra.

Ejemplo: Cliente — Pedido. Un cliente puede tener pedidos.

Línea con diamante relleno (composición)

Indica una relación de todo-parte muy fuerte.

La parte no existe sin el todo.

Ejemplo: Casa ◆— Habitación.

Si la casa desaparece, las habitaciones dejan de tener sentido dentro de ese modelo.

Modificadores: ¿En qué casos usarías un método protected en lugar de private?

Usaría un método protected cuando quiero que el método sea accesible dentro de la clase y también por sus subclases, pero no por código externo.

private

El método solo puede usarse dentro de la misma clase. Es ideal para lógica interna que no debe heredarse ni exponerse.

Ejemplo

Si encenderMotor() fuera private, la subclase Auto no podría reutilizarlo.

protected

El método puede usarse dentro de la clase y por clases que hereden de ella. Es útil para permitir personalización o extensión en subclases.

Ejemplo

en una clase Animal, un método protected respirar() podría ser reutilizado o sobrescrito por subclases como Perro o Pez, mientras que no tendría sentido que cualquier clase externa lo invoque directamente.

Abstracción: Describe cómo abstraerías un "Carrito de Compras" para un ecommerce: ¿Qué datos serían esenciales y cuáles descartarías?

Al abstraer un Carrito de Compras, me enfoco en la información y comportamientos esenciales para el proceso de compra, ignorando detalles irrelevantes.

Datos esenciales

idCarrito: identificador único.

usuarioId: propietario del carrito.

productos: lista de productos con cantidad y precio.

total: monto acumulado.

fechaCreacion y/o fechaActualizacion.

estado (activo, abandonado, comprado, etc.).

Comportamientos clave

agregarProducto()

eliminarProducto()

actualizarCantidad()

calcularTotal()

vaciarCarrito()

Datos que descartaría en la abstracción inicial

Color del botón “Comprar”.

Animaciones de la interfaz.

Resolución de imágenes de productos.

Detalles de seguimiento de clics o métricas de marketing.

Configuraciones específicas del navegador del usuario.